Dashboard / My courses / PL2021 / Final Exam - track A students / FE - part 1

Quiz navigation

1 2 3 4 5 6 7 8 9 10

Finish attempt ...

Time left **0:19:38**

Question **1**

Not yet answered

Marked out of 1.00

Flag question

Which of the answers from below is correct and the most specific?

The list `[A,B|_]` may consist of ...

Select one:

- a. exactly 3 elements
- b. exactly 2 elements
- c. at least 3 elements
- d. at least 2 elements

Dashboard / My courses / PL2021 / Final Exam - track A students / FE - part 1

Quiz navigation

1 2 3 4 5 6 7 8 9 10

Finish attempt ...

Time left **0:17:58**

Question **2**

Not yet answered

Marked out of 1.00

Flag question

In a search operation in an incomplete structure the following condition must hold true:

Select one:

- a. have a clause to check when reached a final variable: `fail,!,var(V).`
- b. have a clause to check when reached a final variable: `var(V),!,fail.`
- c. have a clause to check when reached a final variable: `!,var(V),fail.`
- d. have a clause to check when reached a final variable: `var(V),fail,!.`

Clear my choice

Dashboard / My courses / PL2021 / Final Exam - track A students / FE - part 1

Quiz navigation

1 2 3 4 5 6 7 8 9 10

Finish attempt ...

Time left **0:17:49**

Question **3**

Not yet answered

Marked out of 1.00

Flag question

The following sequence is assumed to search for a key in an incomplete BST. Choose the most specific answer from below:

```
search(t(_,Key,_),Key):-!.
search(t(L,Key1,_),Key):-
  Key1>Key,!,
  search(L,Key).
search(t(_,Key1,R),Key):-
  search(R,Key).
search(T,_):-var(T),!,fail.
```

Select one:

- a. The predicate is incorrect as it is missing the condition `Key1<Key,!,` in the 3rd clause.
- b. The predicate is incorrect. The last clause should be the first one.
- c. The predicate is incorrect, as the fact should be the second clause (search in inorder).
- d. The predicate is incorrect as the last clause should be with `!,var(T),fail.` in the body.

Dashboard / My courses / PL2021 / Final Exam - track A students / FE - part 1

Quiz navigation

1 2 3 4 5 6 7 8 9 10

Finish attempt ...

Time left **0:12:52**

Question **4**

Not yet answered

Marked out of 1.00

Flag question

Fill in the gaps below such as to obtain a predicate which swaps adjacent numbers in a list, if their order is incorrect (the first larger than the second). Use H whenever you need a NEW variable for an element from the list:

one_pass([H1,H2|T], [H1,H2|R]):- H1=<H2, !, one_pass(T , R).

one_pass([H1,H2|T], [H2|R]):-one_pass([H1|T], R).

one_pass([H] , [H]).

one_pass([], []).

?- one_pass([3,1,4,5,2], R).

R=[1,3,4,2,5];

no.

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:12:19**

Question **5**

Not yet answered

Marked out of 1.00

Flag question

Which categories of nodes are frozen (prevented from backtracking) by a cut:

Select one:

- a. The parent node if the cut is the last subgoal
- b. Any subtree of a sibling node to its left after the execution of the cut
- c. Any node to its left before the execution of the cut
- d. Any sibling node to its left at any times

Clear my choice

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:08:36**

Not yet answered

Marked out of 1.00

Flag question

The predicate below is assumed to remove **one** occurence of the first argument, from the list given in the second argument, to generate the list in the third argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
delete(X,[X|T],T).
delete(X,[H|T],[H|R]):-
     delete(X,T,R).
?- delete(b,[a,b,c,b,d],R).
R= [a,c,b,d]? ;
R= [a,b,c,d]? ;
R= [a,b,c,b,d]? ;
no
```

Select one:

- a. Remove the clause delete(X,[X|T],T).
- b. Update the clause 2 as delete(X,[H|T], R):-!,delete(X,T,R).
- c. Update the clause 1 as delete(X,[X|T],T):-!.
- d. Update the clause 1 as delete(X,[X|T],R):-delete(X,T,R).
- e. Update the clause 2 as delete(X,[H|T],[H|R]):-!,delete(X,T,R).
- f. Swap the order of clauses 1 and 2.
- g. Add the clause: delete(_,[],[]).
- h. Add the clause: delete(X,[X],[]).

Clear my choice

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:08:04**

Question **7**

Not yet answered

Marked out of 1.00

Flag question

The following predicate is assumed to check if the tree is BST (Binary Search Tree):

```
is_BST(nil).
is_BST(t(t(LL,KL,LR),K,nil):-
  KL<K,!,
  is_BST(t(LL,KL,LR)).
is_BST(t(t(LL,KL,LR),K,t(RL,KR,RR)):-
  KL<K<KR,!,
  is_BST(t(LL,KL,LR)),
  is_BST(t(RL,KR,RR)).
```

Choose the most specific (and correct) answer from below:

Select one:

- a. the predicate is incorrect, as the inequality check should consider other keys
- b. the predicate is correct
- c. the predicate is incorrect, as it is missing a symmetric clause of the second one with recursive call on the right
- d. the predicate is incorrect as there is no failure clause

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:03:08**

Question **8**
Not yet answered
Marked out of 1.00
Flag question

I want to write a predicate which computes the maximum key in a BST. Complete the partial implementation so that you get a correct predicate:

max(t(K,_,nil,K),K).

max(t(K,L,R),M):-max(R,M).

Next page

◄ Prep coloq en | Jump to... | FE - part 2 ►

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:00:34**

Question **9**
Not yet answered
Marked out of 1.00
Flag question

In a decreasing sorting by selection strategy (below), the `max_delete` predicate is a stable predicate which correctly deletes the maximal value from the list. If we print `Out`, what would be the second printed list on input `In =[3,2,5,1,5]`?

```
sort(In,[M|Out]):-
  max_delete(M,In,Int),
  sort(Int,Out).
sort([],[]).
```

Select one:

- a. [1,2]
- b. [5] ●
- c. [5,5]
- d. [1]

Clear my choice

Quiz navigation

1 2 3 4 5 6
7 8 9 10

Finish attempt ...

Time left **0:00:07**

Question **10**
Not yet answered
Marked out of 1.00
Flag question

Which of the following assert sequences **CANNOT** generate the given clause order for the dynamic predicate `insect/1`:

```
insect(ant).
insect(bee).
insect(moth).
insect(fly).
insect(mosquito).
```

Select one:

- a. **assert**(insect(moth)) -> **asserta**(insect(bee)) -> **asserta**(insect(ant)) -> **assert**(insect(fly)) -> **assert**(insect(mosquito))
- b. **asserta**(insect(moth)) -> **asserta**(insect(bee)) -> **assertz**(insect(fly)) -> **assert**(insect(mosquito)) -> **asserta**(insect(ant))
- c. **asserta**(insect(mosquito)) -> **asserta**(insect(fly)) -> **asserta**(insect(moth)) -> **assert**(insect(bee)) -> **asserta**(insect(ant)) ●
- d. **asserta**(insect(bee)) -> **assert**(insect(moth)) -> **asserta**(insect(ant)) -> **assert**(insect(fly)) -> **assert**(insect(mosquito))

Clear my choice

Quiz navigation

1 2 3 4 5 6 7 8 9

Finish attempt ...

Time left **0:23:47**

Question **1**

Not yet answered

Marked out of 1.00

Flag question

Given the **correct** `minwithdelete/3` predicate below for selecting and deleting the minimum element from a list, when running the sequence with the specified input, which is the output list obtained?

```
minwithdelete([H|T],Minim,MinDeleted):-
      min_del(T,H,Minim, MinDeleted).
min_del([H|T],MP,MF,[MP|R]):-
      H<MP,!,
      min_del(T,H,MF,R).
min_del([H|T],MP,MF,[H|R]):-
      min_del(T,MP,MF,R).
min_del([],M,M,[]).
```

On input list `[3,4,2,5,7,1]` the output MinDeleted will be (choose one):

Select one:

- a. [3,4,2,5,7,1]
- b. [3,4,5,7,2]
- c. [4,3,5,7,2]
- d. [3,4,2,5,7]

Clear my choice

ashboard / My courses / PL.2021 / Final Exam - track A students / FE - part 2

uiz navigation

1 2 3 4 5 6 7 8 9

ish attempt ...

me left **0:15:17**

Question **2**

Not yet answered

Marked out of 1.00

Flag question

You are given an ***incomplete*** binary tree. The predicate `transform/2` from below should replace all keys in leaf nodes, with -1. Complete the predicate specification accordingly:

```
transform(T,_):-
-1 .
transform(t(_,T,T), t(-1,_,_)):-
 .
transform(t(K,L,R),
 ):-
    transform( ,NL),
    transform( ,NR).
```

iiz navigation

2 3 4 5

7 8 9

ish attempt ...

ıe left **0:05:07**

Question **4**

Not yet answered

Marked out of 1.00

Flag question

The following sequence is assumed to decreasingly order the list provided as first argument.

```
sort_decreasing(In,Out):-
      helper(X,[A,B|Y],In),
      A<=B,!,
      helper(X,[B,A|Y],Int),
      sort_decreasing (Int,Out).
sort_decreasing(In,In).

helper([],L,L).
helper([H|T],L,[H|R]):-
      helper(T,L,R).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is incorrect as it returns the input list; in clause 2 of `sort_decreasing/2`, the output unifies with the input.
- b. The sequence is incorrect, as because of the ! after the inequality, the non-determinism cannot apply.
- c. The sequence is correct, ordering decreasingly an input list, provided it contains no duplicates.
- d. The sequence is correct, ordering decreasingly any input list regardless of the presence of duplicates

Quiz navigation

1 2 3 4 5 6

7 8 9

Finish attempt ...

Time left **0:03:54**

Question **5**

Not yet answered

Marked out of 1.00

Flag question

Given an *incomplete* BST (Binary Search Tree ended in variable), the predicate below should collect in an ordered list the keys in nodes at a **given** height.

```
collect_IT(T,0,_,[]):-var(T),!.
collect_IT(t(L,K,R),H,GivenHeight,CollectList):-
      collect_IT(L,HL,GivenHeight,CollectLeft),
      collect_IT(R,HR,GivenHeight,CollectRight),
      max(HL,HR,HI),
      H is HI+1,
      build_list(CollectLeft,CollectRight,CollectList,K,H,GivenHeight).

build_list(LL,RL,Result,_,H,H):-!,
      append(LL,RL,Result).
build_list(LL,RL,Result,Key,_,_):-!,
      append(LL,[Key|RL],Result).
```

Choose the most specific correct answer from below:

Select one:

- a. The predicate is incorrect as it does not check the height when the final variable is reached.
- b. The predicate is incorrect as it does not add the Key in the root when it should (should be added in the first clause of `build_list` instead of the second one).
- c. The predicate is incorrect because the height is incorrectly calculated.
- d. The predicate is correct as it is as it adds the keys at the specified height and concatenates them orderly.

Clear my choice

Quiz navigation

1 2 3 4 5 6
7 8 9

Finish attempt ...

Time left **0:03:02**

Question **6**

Not yet answered

Marked out of 1.00

Flag question

The `replace_duplicates` predicate below replaces the duplicate elements in a list (the elements which appear more than once), with a new given element, `El`.

```
replace_duplicates([H|T],El,[El|R]):- m(H,T),!,
                                      r(H,El,T,RI),
                                      replace_duplicates(RI,El,R).
replace_duplicates([_|T],El,R):-replace_duplicates(T,El,R).
replace_duplicates([],_,[]).

r(_,[],[]).
r(H,El,[H|T],[El|R]):- r(H,El,T,R).
r(X,El,[H|T],[H|R]):- r(X,El,T,R).

m(X,[X|_]):- !.
m(X,[_|T]):- m(X,T).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is incorrect, because the `r/4` predicate is non-deterministic.
- b. The sequence is correct as `replace_duplicates/3` has a non-deterministic behavior.
- c. The sequence is incorrect, as the [El|R] pattern should be in the head of the second clause of predicate replace_duplicates/3, not the first.
- d. The sequence is correct as it is, because it replaces all the other occurrences when encountering a duplicate.

Quiz navigation

1 2 3 4 5 6
7 8 9

Finish attempt ...

Time left **0:02:23**

Question **7**

Not yet answered

Marked out of 1.00

Flag question

The `count_duplicates` predicate below counts the duplicate elements in a list (the elements which appear more than once).

```
count_duplicates([H|T], C):-    m(H,T),
                                d(H,T,RI),
                                count_duplicates(RI,C1),
                                C is C1+1.
count_duplicates([_|T], C):- count_duplicates(T,C).
count_duplicates([],0).

d(_,[],[]).
d(H,[H|T],R):- !, d(H,T,R).
d(X,[H|T],[H|R]):- d(X,T,R).

m(X,[X|_]):- !.
m(X,[_|T]):- m(X,T).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is correct, as it has a clause for non-duplicate elements.
- b. The sequence is incorrect, as the last clause of `count_duplicates/2` must be put as the first clause.
- c. The sequence is incorrect, as it has a non-deterministic behavior for counting (i.e. a ! has to be placed in the first clause of `count_duplicates/2`, after the call `m(H,T)`).
- d. The sequence is correct as it is, because it deletes all the other occurrences when counting a duplicate.

Quiz navigation

1 2 3 4 5 6
7 8 9

Finish attempt ...

Time left **0:00:35**

Question **9**

Not yet answered

Marked out of 1.00

Flag question

Given a complete BST (Binary Search Tree) the predicate below should collect in an ordered list the keys in nodes at a **given** level, using **difference lists**.

```
collect_DL(nil,_,_,CollectedList, CollectedList).
collect_DL(t(_,K,_),Level,Level,[K|CollectedList],CollectedList):-!.
collect_DL(t(L,_,R),Level,GivenLevel,CollListBeg,CollListEnd):-
      Level1 is Level+1,
      collect_DL(L,Level1,GivenLevel, CollListBeg,Int),
      collect_DL(R,Level1,GivenLevel,Int, CollListEnd).
```

Choose the most specific correct answer from below:

Select one:

- a. The predicate is correct as it is as it adds the keys at the specified level and concatenates them orderly.
- b. The predicate is incorrect as it does not check the level on `nil`.
- c. The predicate is incorrect as it ignores the left and right subtrees in the second clause.
- d. The predicate is incorrect as it ignores the key from the root in the third clause.

Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left **0:29:13**

Question **1**

Not yet answered

Marked out of 1.00

Flag question

Given a knowledge base of the form `weather(city_name,temperature).`, you must write a predicate to collect in a list the names of all cities with a temperature over a given threshold (temperature >= threshold), WITHOUT using the `findall` predicate. Complete the `get_all_temp/2` predicate specification below accordingly:

```
get_all_temp(Th, _):- weather(City,T),
                      [          ],
                      assert([          ]),
                      [          ].
get_all_temp(_,L):-[          ].
g([          ]):-retract(good(H)),
                 [          ],
                 g(R).
g([          ]).
```


Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left **0:28:59**

Question **2**

Not yet answered

Marked out of 1.00

Flag question

The predicate `is_euler` from below checks whether a connected graph (specified in the edge-clause form) has an Euler cycle (circuit), or not.

```
compute_degree(X,D):-  setof(Y, (edge(X,Y);edge(Y,X)), Neigh),
                       !,
                       length(Neigh, D).
compute_degree(_,0).

is_euler:- (edge(X,_);edge(_,X)),
           not(seen(X)),
           assert(seen(X)),
           compute_degree(X,D),
           not(0 is D mod 2),
           !,
           is_euler.
is_euler.
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is correct, as it checks, for each vertex, that it has an even degree.
- b. The sequence is incorrect, as it should contain a `fail` after ! (instead of a recursive call).
- c. The sequence is correct, but it computes the degree of a node several times (for each edge it appears in).
- d. The sequence is incorrect, the ! in the first clause of `is_euler` should be removed to make it correct.

Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left **0:28:30**

Question **3**

Not yet answered

Marked out of 1.00

Flag question

Given the edge representation (`edge(X,Y,W)`) of an undirected graph, the predicate `exists_with_same_weight` below checks if **some** (any) vertex has 2 edges with the same weight:

```
exists_with_same_weight:-
      is_edge(X,Y,W),
      is_edge(X,Z,W),
      Y \= Z,
      assertz(same_weight(X,Y,Z,W)).
exists_with_same_weight:-
      writeln("no such vertex exists").

is_edge(X,Y,D):-
      edge(X,Y,D);
      edge(Y,X,D).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is correct as the entire graph is scanned.
- b. The sequence is incorrect as not the entire graph is scanned.
- c. The sequence is incorrect because it is missing a ! after Y \= Z.
- d. The sequence is correct as it scans the graph until the condition is met.

Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left **0:28:17**

Question **4**

Not yet answered

Marked out of 1.00

Flag question

Given the edge representation (`edge(X,Y,W)`) of a weighted undirected graph, we want to verify that there is **no** vertex in the graph having an edge with a **given** weight. Given the specification:

```
no_given_weight(W):-
      is_edge(X,_,W),!,
      fail.
no_given_weight(D):-
      writeln("no vertex has an edge with weight D").
```

choose the most specific correct answer from below:

Select one:

- a. The sequence is incorrect because the `!` after `is_edge(X,_,W)` prevents the backtracking mechanism to scan the entire graph.
- b. The sequence is incorrect as the second clause in `given_weight` has a different argument than the first clause.
- c. The sequence is incorrect because of the `fail`; we should have had a recursive call instead.
- d. The sequence is correct as the entire graph is scanned and for each edge, the weight is compared with the given weight.

Quiz navigation

1 2 3 4 5

Finish attempt ...

Time left **0:00:26**

Question **5**

Not yet answered

Marked out of 1.00

Flag question

Given the edge representation of an undirected graph, the predicate `difference_graph` below generates the difference graph Gdiff of G1 and G2 (assume a predicate `is_edge` is present for each 3 graphs - G1 G2 and Gdiff):

```
difference_graph:-
      is_edge_1(X,Y),
      not(is_edge_2(X,Y)),
      not(is_edge_diff(X,Y)),
      assertz(edge_diff(X,Y)),
      fail.
difference_graph.

is_edge(X,Y):-
      edge(X,Y);
      edge(Y,X).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is incorrect, as it is missing a ! after `not(is_edge_diff(X,Y))`.
- b. The sequence is correct as both G1 and G2 are entirely scanned once.
- c. The sequence is incorrect, as it is missing a ! after `not(is_edge_2(X,Y))`.
- d. The sequence is correct as G1 is entirely scanned and G2 is scanned for each edge in G1.

Clear my choice